OS project report: Restaurant

Elia Tisato - 252305
Davide Zarantonello - 254061
Kingsley Okai - 253956

Contents

Disclaimer: Instructions say that this report must be no longer than 5 pages. To be correct, this PDF is 8 pages long, but the effective content is 4 and a half pages long.

Section 1.
Introduction

In this project, we've implemented a simulation of a restaurant featuring customers, waiters, and cooks, each running on a separate thread. The simulation outputs the restaurant's score, based on client satisfaction, and a result indicating whether errors occurred and which ones.

In the following paragraphs, there will be explanations of the various parts of our project.

Section 2.
Architecture

First of all, we need to explain the source tree structure. All the modules are in the `lib/` folder, with modules for the utilities at the root:

- `vec` is just a dynamic array;
- `fifo-queue` is a dynamically allocated singly linked list;
- `rng` is an implementation of the xoshiro256++ [1] algorithm, with some useful functions for initialising the main and per thread RNGs;
- `result` simply contains an `enum` with all the result codes.

Into `lib/data` are stored all the modules needed for loading dishes, requirements and resources from CSV files and into `lib/state` there are modules needed for coordinating threads, such as `Kitchen`, `Sink`, `Restaurant` and `DishTicket`. Finally, into `lib/threads` we put modules that manage the threads: `Cook`, `Waiter` and `Customer`.

The `wrapper.c` in `lib/threads` contains functions to safely initialise a thread and return the thread process result.

2.1. Shared data structures

There are many shared data structures, all protected by `pthread` mutexes. When any thread wants to read or write the shared values, it must lock the mutex, do the operation and then unlock.

- `Restaurant`: the score, the `FIFOQueue` of all spawned customers (ones that left the restaurant have `has_left=true`) and some customer counters are protected by a single mutex since they are all related data that on multiple occasions needs to be updated together. The atomic `bool` variable `is_closing` is used to indicate to waiters and cooks that the program is terminating. Other fields like the waiters and cooks vectors aren't protected since once initialised they become read-only until drop;
- `KitchenResources`: the atomic `bool available` indicates if the resource can be obtained or not. `dirtiness` didn't need a mutex since it's only updated while a single cook thread holds the resource.

In addition, each thread has its own mutex. In particular:

- `Cook` has a mutex protecting `FIFOQueue` of `DishTicket` updated by waiters via the `cook_assign` function, and some counters read by waiters;
- `Waiter` has a mutex protecting `FIFOQueue` of ready `DishTicket` updated by cooks via the `waiter_assign` function;
- `Customer` has a mutex protecting some counters and `bool` variables, used by waiters for serving dishes and scheduling the order.

Speaking of threads, cooks and waiters also have a semaphore:

- The cooks' one indicates that there are dishes to cook, if not they do nothing;
- The waiters' one indicates that they have dishes to serve, otherwise they get orders or try to entertain the customer with the least amount of patience left.

2.2. Configuration loading

The `bootstrap.sh` file has the job of loading the configuration from the env file and, optionally, from command line arguments. It validates every option with regexes and also tries to check that the content of the CSV files is what's expected by the `restaurant` executable.

The env file must always be valid and contain all the parameters, even if one of those is later overridden by a CLI argument. For simplicity, we decided to pass all the arguments loaded and validated from `bootstrap.sh` into environment variables.

Parsing the CSV files required the handling of arbitrary user strings, so instead of fixed length buffers we opted to build the `Vec` type from scratch to simplify dynamic allocation.

`str.c` contains two functions that read strings until a character is met, and return the consumed length. We opted for this approach instead of something like `sscanf` to avoid all the edge cases created by whitespace and malformed content parsing that we would have needed to handle.

Once `Dishes` and `Resources` are loaded, they stay fixed in memory, allowing every other part of the program to refer to them by their pointer value instead of having to do `strcmp` on their names.

Section 3.
Design choices

3.1. Acquiring kitchen resources

We found the `Kitchen` to be the most challenging element of the project.

For the cooks, we settled on looping continuously over the queue of tickets assigned by the waiters. If the ticket's dish has readily available resources, then the cook acquires them and cooks it; if it doesn't, then we back off for the duration of an in-game tick and then retry the loop.

In the initial version of the `Kitchen` (fundamentally an array of resources that keeps track of its availability and dirtiness), we used a global mutex over the array of resources, with an all or nothing acquisition algorithm, but the contention over that lock made the program stutter.

Our final improvements focused on resolving the performance impact caused by the cook threads:

- first we switched to an algorithm that doesn't require a lock over the kitchen. Now, through atomic variables, each cook tries to acquire the needed resources sequentially by setting `kitchen_resource->available` to `false` (the order is fixed to avoid deadlocks) and if it fails, it rolls back immediately;
- then the problem became that cooks were trying and failing so often to acquire random resources that they also penalised the others. We solved this by implementing a policy of acquiring resources only for the plate with the least contended resources. Now we calculate a contention score for each plate in the list and try only the one with the highest probability to succeed.

3.2. The sink

The `Sink` allows cooks to wash their resources. The requirement that only one cook can use it at a time is satisfied by locking the 'washing' behind a semaphore.

The washing is done only if retained necessary after the cook has completed the preparation of the plate and given it to the waiter. Scheduling when to wash a resource is made through a formula that compares the score that the restaurant would lose on the next use if continuing without washing, versus the score that could be gained from the customer with the time gained from not washing.

$$\text{wash} \Leftrightarrow \left(\text{clean\_time} * (\text{waiting} + 1) * \frac{\text{queued\_price}}{\max(\text{queued\_time}, 1)} < 2^{\text{dirtiness}} * \log_2(1 + \text{clean\_time})\right)$$

We found it to work quite well as the sink is not a point of contention as much as the queues for the resources are.

3.3. Customers and score

The customer works by continuously checking in a loop if its `time_waiting` has reached its `patience` limit.

`Customer` objects are spawned in the main thread after a random delay. All of them are allocated in memory and never moved from there, since we would have no way to update the pointer passed to the thread. They remain in the queue as long as all the dish tickets have been given back to them (even if they already left the restaurant). This is done to ensure that waiters never access memory that would have been freed by dropping the item from the queue.

To avoid the queue becoming too long, we implemented a periodic cleanup mechanism that, when it's safe (customer left and no pending tickets), pops the first customer from the queue, freeing up some memory.

The score of the restaurant is kept in a global `int` variable. We considered making it an atomic variable but later we discovered that we had to update other fields together so now it's protected by the restaurant's `mtx`. This may have a performance impact since that was already a point of contention between waiters needing to check the customers' queue to take orders and the `restaurant_spawn_customer` function.

3.4. Order scheduling (waiter)

To determine which cook to assign an order to, the waiter uses a formula which is a sum of two shares:

- the first (own_cost) takes account of the ratio between the total time the cook will cook and the customer's remaining time in the restaurant and its weighted by the price so that an expensive plate is lean to be done first, this share is multiplied by two when the expected time to finish all plates is greater than the time left before the customer leaves, this is to plummet the probability that the dish is given to that cook, but it's not entirely discarded as the cook may decide to cook the dish first and the waiter could successfully entertain the client;
- the second (others_cost) considers the cook's queue value per unit of time multiplied by the time it takes to cook the plate: a cook with a high value here should not be disturbed as in the time it would have taken to cook the dish, he will have prepared other dishes for a huge revenue.

After the calculations, the waiter assigns the dish to the cook with the lowest sum.

The max functions are used as both customer_patience — customer_time_waiting and queued_time can be 0 and are the denominator to a fraction.

$$\text{own\_cost}_c = \frac{\text{order\_total\_price} * (\text{queued\_time}_c + \text{dish\_cook\_time})}{\max(\text{customer\_patience} - \text{customer\_time\_waiting}, 1)}$$

$$\text{own\_cost} = \text{own\_cost} * 2 \text{ if cook can't finish in time}$$

$$\text{others\_cost}_c = \text{dish\_cook\_time} * \frac{\text{queued\_price}_c}{\max(\text{queued\_time}_c, 1)}$$

$$\text{assign to} \min_c(\text{own\_cost}_c + \text{others\_cost}_c)$$

3.5. Waiter entertainment

Waiters with nothing to do can (with the set probability) try to entertain a customer. While we couldn't choose the amount of patience added or removed to the customer, we programmed waiters to choose to entertain the customer with the least amount of patience left, in order to minimise the damage in case of a loss of patience and to possibly give the time needed to complete the last dishes in case of a positive change.

3.6. Simulated time

Each thread manages its own sleeping times, with respect to the globally configured `GAME_SPEED`. As requested, we used the `usleep` function.

Time is measured in ticks where `n` ticks are $\frac{1*10^6*n}{\text{GAME\_SPEED}}$ microseconds.

3.7. Reproducibility

The xoshiro256++ algorithm is quite suitable for multi-threaded programs like this one. We initialise a main `RNGState` that always outputs the same sequence of numbers for the same seed.

Each thread is assigned a copy of the main state in a synchronous manner (inside the spawn function run from main) and the main state is jumped by 2^{128}. This allows to avoid the need for synchronisation primitives completely since each thread state becomes independent.

With this method, we can ensure that random numbers are generated reproducibly; however, especially at high speeds, the program is not fully reproducible, since discrepancies between the scheduling of semaphores and threads done by the OS can alter the score returned by some formulas, introducing deviations in the final score of the program.

3.8. Error handling

Error handling was a tricky part due to the limitations of C's type system. We settled on a global `Result enum` with all the possible error codes, kept in sync with the environment variables exported from `result.sh`. Each function that can fail returns an error code or `RESULT_OK` otherwise.

Having to handle all the possible failures required duplicating the cleanup logic in various places. To avoid cluttering up the code with all the drop functions, we decided to use `goto cleanup;` statements to jump to the cleanup section. A convention endorsed also by the Linux kernel [2].

Section 4.
Verification and optimizations

The program was developed with severe discipline and using modern tools like the Clang compiler's address, undefined and thread sanitisers [3] and Valgrind [4]. This lets us be confident that memory is handled properly and (at least in the simple cases) race conditions shouldn't be present.

We tried to make it realistic and we avoided allowing cooks to access customer state (like a cook in a real kitchen cannot see its customers), so the biggest optimisation that was left on the table would

have been the cancellation of plates destined to customers that already left (a simple `dish_ticket.customer.has_left` check inside `cook_select_next_ticket`). In our case, the plate is always given to the waiter, who simply increments the `customer.dishes_served` value in order to allow dropping the customer from the queue and free memory early.

Section 5.
Conclusion

The biggest bottleneck that we measured on the final implementation seems to be the scheduling of resources in the `Kitchen`. On longer runs, cooks simply cannot keep up with the amount of orders and the queues get longer and longer.

We spent quite a bit of time trying to tune the various formulas used to calculate scores throughout the program and revising several architecture decisions to try and make the restaurant more successful, but in the end we weren't able to reach a point where the execution would return a positive score.

Bibliography

[1] ‘Xoshiro256++ source code’. [Online]. Available: https://prng.di.unimi.it/xoshiro256plusplus.c

[2] ‘Linux kernel coding style’. [Online]. Available: https://www.kernel.org/doc/html/v7.2/process/coding-style.html#centralized-exiting-of-functions

[3] ‘Compiler-rt 's website'’. [Online]. Available: https://compiler-rt.llvm.org/

[4] ‘Valgrind's website’. [Online]. Available: https://valgrind.org/